

ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

AppleTopia Webshop

2026.

Szakma:
SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ
5 0613 12 03

Készítették:
Stadler Áron
Seres Olivér Ármin
Csöndes Máté János

Tartalomjegyzék

I.	Bevezetés.....	6
II.	Témaválasztás indoklása	6
III.	Fejlesztői dokumentáció, technikai leírás.....	8
1.	Specifikáció	8
1.1	A rendszer áttekintése.....	8
1.2	Funkcionális specifikáció	8
1.2.1	Felhasználói hitelesítés	8
1.2.2	Adatkezelés (CRUD műveletek)	8
1.2.3	API kommunikáció.....	9
1.3	Nem funkcionális specifikáció	9
1.3.1	Teljesítménykövetelmények.....	9
1.3.2	Biztonsági követelmények.....	9
1.3.3	Használhatósági követelmények	9
1.3.4	Karbantarthatósági követelmények	10
1.4	Rendszerarchitektúra specifikáció.....	10
2.	Az alkalmazott fejlesztői eszközök	10
3.	Adatmodell leírása.....	10
3.1	Táblák és leírásuk	11
3.2	Üzleti logika:	12
3.3	Átlagos felhasználói folyamat	12
3.4	Megkötések és integritás	12
4.	Algoritmusok.....	13
4.1	Felhasználói hitelesítés és jelszókezelés.....	13
4.2	Felhasználói szerepkör módosítása	14
4.3	Kosár és rendeléskezelési folyamat.....	15

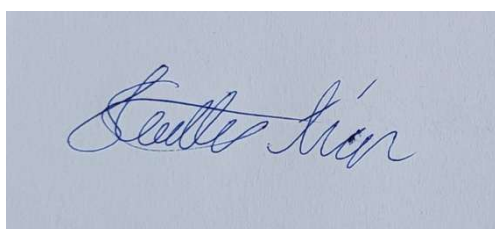
4.4	Rendelés létrehozása és életciklus-kezelés.....	16
5.	Tesztelések.....	18
5.1	Tesztelési stratégia.....	18
5.2	Frontend tesztelés	18
5.3	Tesztelési típusok	18
5.4	Backend tesztelés.....	19
5.5	API tesztelés	19
5.6	Tesztelési eredmények.....	19
IV.	Felhasználói dokumentáció	20
6.	Program általános specifikációja és célja	20
6.1	Program célja.....	20
7.	Általános specifikáció.....	20
7.1	Támogatott funkciók:	20
8.	Rendszer követelmények.....	20
8.1	Szoftver követelmények	20
8.2	Backend követelmények.....	20
8.3	Frontend követelmények	21
8.4	Hardver követelmények.....	21
8.5	Hálózati portok	21
9.	Program telepítése	21
9.1	Előfeltételek ellenőrzése.....	21
9.2	Projekt letöltése	21
9.3	Az alkalmazás indítása	22
9.4	Indítási adatok	22
10.	Program használata.....	22
10.1	Az alkalmazás indítása	22

10.2	Felhasználói fiókok	22
10.2.1	Regisztráció	22
10.2.2	Bejelentkezés	23
10.3	Termékek böngészése.....	24
10.4	Termék részletei	24
10.5	Kosár kezelés	25
10.5.1	Termék hozzáadása a kosárhoz	25
10.5.2	Kosár megtekintése.....	25
10.5.3	Termék eltávolítása a kosárból.....	25
10.5.4	Rendelés leadása.....	25
10.5.5	Rendelések követése.....	27
10.6	Felhasználói fiók kezelése	28
10.7	Admin funkciók (adminisztrátorok számára).....	28
10.7.1	Admin panel elérése	28
10.7.2	Termékek kezelése	29
10.7.2.1	Új termékek hozzáadása	29
10.7.2.2	Meglévő termékek kezelése	29
10.7.3	Specifikációk kezelése.....	30
10.7.4	Rendelések kezelése	30
10.8	Ügyfélszolgálat és Hiba kezelés	31
V.	Összegzés	31
VI.	Irodalomjegyzék, hivatkozásjegyzék	33
VII.	Ábrajegyzék.....	34

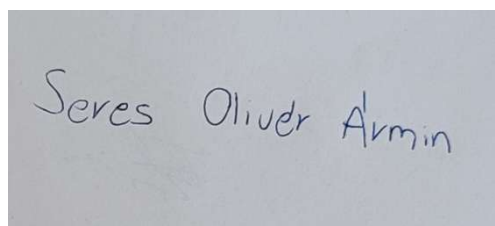
Nyilatkozat

Alulírott Stadler Áron, Seres Olivér Ármin, Csöndes Máté János büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

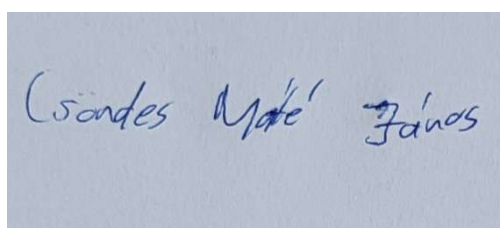
Paks, 2026. április 22.



Stadler Áron



Seres Olivér Ármin



Csöndes Máté János

I. Bevezetés

Napjainkban egyre többen választják az online vásárlást annak gyorsasága, egyszerűsége és számos további előnye miatt. Az AppleTopia webáruház célja, hogy megkönnyítse az Apple telefonok és tabletek beszerzését, valamint modern és felhasználóbarát felületet biztosítson a vásárlók számára.

Webáruházunk arra törekszik, hogy megbízható szolgáltatást nyújtson, és a lehető legkedvezőbb árakat kínálja mindazoknak, akik új vagy korábbi Apple eszközöket szeretnének megvásárolni. A kínálatban helyet kapnak a legfrissebb készülékek mellett a használt és felújított termékek is, így mindenki megtalálhatja az igényeinek és pénztárcájának megfelelő eszközt.

A vásárlási élményt tovább színesítik az időszakos akciók, amelyek segítségével a látogatók még kedvezőbb áron juthatnak hozzá a kívánt termékekhez.

II. Témaválasztás indoklása

A témaválasztásunkat több szempont is indokolta. Egyrészt az online kereskedelem folyamatosan növekszik így egy webáruház elkészítése naprakész és hasznos tudást nyújthat, másrészt a technológiai eszközök – különösen az okostelefonok és táblagépek – kiemelt szerepet töltenek be a mindennapi életben. Egy ilyen típusú weboldal elkészítésével betekintést nyerhetünk a webáruházak felépítésébe, struktúrájába és a háttérfolyamatok alakulásába. Emellett megismerhetjük mi tesz jóvá egy weboldalt, azaz milyen elvárásaik vannak a vásárlóknak, felhasználóknak.

A témához kapcsolódó korábbi kutatások és szakirodalmak elsősorban az online vásárlói viselkedésre, a webes felhasználói élmény optimalizálására, illetve az e-kereskedelmi rendszerek felépítésére fókuszálnak. Ezekből kiemelt hangsúlyt kaptak azok az eredmények, amelyek a letisztult navigáció, az átláthatóság, a biztonságos fizetési rendszerek és a rezponzív tervezés fontosságát emelik ki.

A vizsgaremek elkészítése során olyan eszközöket és eljárásokat alkalmaztunk, amelyek a modern webfejlesztés alapját képezik. A vizsgaremek frontend programozási része React-el készült, azon belül JavaScriptet alkalmaztunk, ezt a Visual Studio Code tette lehetővé. A Backend programozási részéhez ASP.NET-re került a választás, aminek megvalósítására a Visual Studiot alkalmaztuk. A programozásunk folyamatának követésére, és a fájlok tárolására a GitHub-ot alkalmaztuk.

III. Fejlesztői dokumentáció, technikai leírás

1. Specifikáció

1.1 A rendszer áttekintése

A rendszer egy full-stack webalkalmazás, amely C# ASP.NET Core backendből és React alapú frontendből áll.

Az alkalmazás kliens–szerver architektúrát követ, ahol a frontend REST alapú HTTP kéréseken keresztül, JSON formátumban kommunikál a backenddel.

A backend felelős az üzleti logika megvalósításáért, az adatellenőrzésért, a hitelesítésért és az adatbázis-kezelésért.

A frontend biztosítja a felhasználói felületet, kezeli a kliensoldali állapotot és az API-hívásokat.

1.2 Funkcionális specifikáció

1.2.1 Felhasználói hitelesítés

A rendszernek biztosítani kell:

Új felhasználók regisztrációját.

Bejelentkezési lehetőséget érvényes hitelesítő adatokkal.

A felhasználói adatok biztonságos ellenőrzését.

A jelszavak titkosított (hash-elt) tárolását.

A védett végpontokhoz való hozzáférés korlátozását nem hitelesített felhasználók számára.

1.2.2 Adatkezelés (CRUD műveletek)

A rendszernek lehetőséget kell biztosítani az alábbi műveletekre:

Új rekord létrehozása.

Létező rekord lekérdezése.

Rekord módosítása.

Rekord törlése.

Minden művelet esetében biztosítani kell:

Szerveroldali adatellenőrzést.

Megfelelő HTTP státuszkód visszaadását.

Hibakezelést és egyértelmű hibaüzenetek megjelenítését.

1.2.3 API kommunikáció

A rendszernek:

- REST alapú API végpontokat kell biztosítani.
- JSON formátumot kell használnia adatcserére.
- Az alábbi HTTP státuszkódokat kell alkalmaznia:
 - 200 OK
 - 201 Created
 - 400 Bad Request
 - 401 Unauthorized
 - 404 Not Found
 - 500 Internal Server Error

1.3 Nem funkcionális specifikáció

1.3.1 Teljesítménykövetelmények

Az API válaszüzeje normál terhelés mellett nem haladhatja meg a 2 másodpercet.

Az adatbázis-lekérdezéseket optimalizálni kell a felesleges terhelés elkerülése érdekében.

1.3.2 Biztonsági követelmények

A kommunikációnak HTTPS protokollon keresztül kell történnie.

A jelszavakat titkosított formában kell tárolni.

Az adatbevitel ellenőrzésének meg kell akadályoznia az injekciós támadásokat.

A rendszernek hitelesítési és jogosultságkezelési mechanizmust kell alkalmaznia.

1.3.3 Használhatósági követelmények

A felhasználói felületnek reszponzív kialakításúnak kell lennie.

Az alkalmazásnak korszerű webböngészőkben működni kell.

A hibaüzeneteknek egyértelműnek és érthetőnek kell lenniük.

1.3.4 Karbantarthatósági követelmények

A backendnek rétegzett architektúrát kell követnie.

A frontendnek újrafelhasználható komponenseket kell alkalmaznia.

A forráskódnak strukturáltnak és dokumentáltnak kell lennie.

1.4 Rendszerarchitektúra specifikáció

A rendszer háromrétegű architektúrát alkalmaz:

Megjelenítési réteg (React Frontend)

Alkalmazási réteg (ASP.NET Core Web API)

Adatkezelési réteg (Relációs adatbázis)

A frontend kizárólag HTTP kéréseken keresztül kommunikál a backenddel.

A backend feldolgozza a beérkező kéréseket, alkalmazza az üzleti szabályokat, majd egy ORM (pl. Entity Framework Core) segítségével kommunikál az adatbázissal.

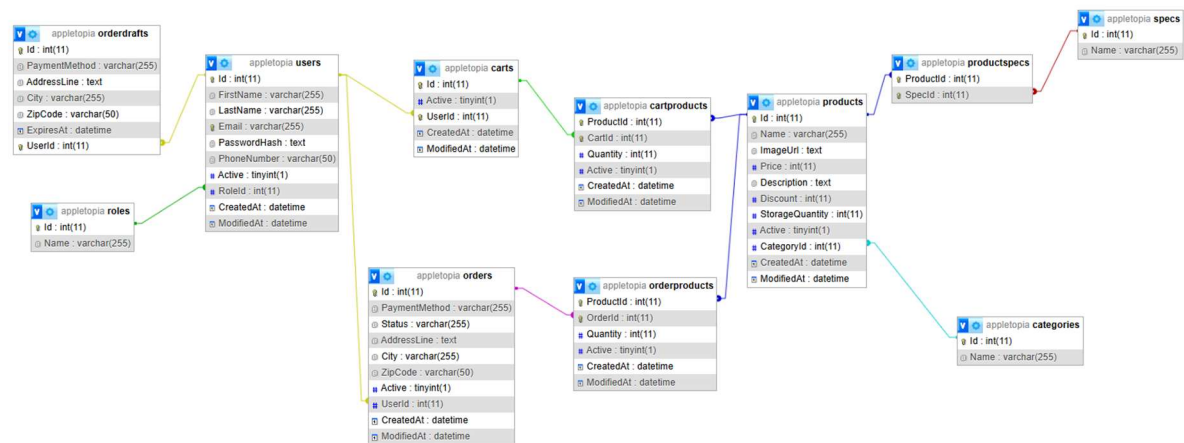
2. Az alkalmazott fejlesztői eszközök

- **Programozási nyelvek:** Typescript, C#
- **Frontend Framework:** React 19.2.0
- **Backend Runtime:** ASP.NET Core 8.0
- **Adatbázis:** SQLite
- **Verzió kezelő:** Git (GitHub repository)
- **Fejlesztői környezet:** Visual Studio Code on Windows 11, Visual Studio 2026

3. Adatmodell leírása

Az adatbázis egy webshop rendszert modellez, amely tartalmazza:

- Felhasználók és szerepkörök kezelése
- Termékkatalógus kategóriákkal és specifikációkkal
- Kosár rendszer
- Rendeléskezelés
- Rendelési pizskozat (checkout folyamatnál)



1. ábra Adatbázis felépítése

3.1 Táblák és leírásuk

Roles: Adott felhasználó jogosultságát határozza meg (admin vagy user)

Users: A rendszer felhasználói (egy Role → több User, egy User → egy Role)

Category: Egy adott termék kategóriáját határozza meg (admin tudja felvenni)

Products: A rendszer termékei (egy Category → több Product, egy Product → egy Category)

Specs: Termékek specifikációja (admin tudja felvenni)

ProductSpecs: Kapcsoló tábla (több – több kapcsolat a Products és Specs között)

Carts: Adott felhasználó kosara (egy Cart → egy User)

CartProducts: Kapcsoló tábla (több – több kapcsolat a Cart és Products között)

Orders: Adott felhasználó rendelései (egy User → több Orders, egy Orders → egy User)

OrderProducts: Kapcsoló tábla (több – több kapcsolat Orders és Products között)

OrderDraft: Rendelés piszkozata (egy User → egy Orderdraft)

Kapcsolatok összefoglalása:

- User → Role (több az egyhez)
- User → Cart (egy az egyhez)
- User → Orders (egy a többhöz)
- User → OrderDraft (egy az egyhez)
- Category → Products (egy a többhöz)

- Products → Specs (több a többhöz)
- Cart → Products (több a többhöz)
- Orders → Products (több a többhöz)

3.2 Üzleti logika:

- Egy felhasználónak egy kosara és rendelési pizkozata van
- Egy termék egy kategóriába tartozik és több specifikációja van
- A rendelés a végleges vásárlást jelenti
- A rendelési pizkozat az egy ideiglenes checkout állapot, ami lejár az „ExpiresAt” alapján

3.3 Átlagos felhasználói folyamat

- Felhasználó regisztrál → kap egy User role-t, létrejön a kosara
- Termékeket hozzáad a kosárhoz → kosárban megjelenik
- Checkout indítása → szállítási adatok megadás
- Megrendelés véglegesítése, kosár ürítése
- Rendelések a felhasználó profilján megtekinthetők

3.4 Megkötések és integritás

- Felhasználó törlése törli a kosarat, rendeléseket és a pizkozatot
- Egyedi mezők:
 - Users.Email
 - Carts.UserId
 - Minden tábla Id-je
 - OrderDrafts.UserId
- Összetett kulcsok
 - CartProducts (CartId, ProductId)
 - OrderProducts (OrderId, ProductId)
 - ProductSpecs (ProductId, SpecId)

4. Algoritmusok

Az alkalmazás működésének alapját több kulcsfontosságú algoritmus és folyamat alkotja. Az alábbiakban bemutatjuk az egyes funkciók mögött álló logikát és megvalósítási módszereket.

4.1 Felhasználói hitelesítés és jelszókezelés

A hitelesítési folyamat két részből áll: regisztrációból és bejelentkezésből. Az AuthController kezeli mindkét végpontot ([HttpPost("register")] és [HttpPost("login")]). A jelszókezeléshez az ASP.NET Core Identity beépített PasswordHasher<object> osztályát alkalmazzák, a védett végpontok JWT tokennel védettek.

Regisztrációs folyamat:

- A felhasználó megadja a keresztnévét, vezetéknévét, e-mail címét, jelszavát és telefonszámát (UserRegister DTO). A rendszer nem ellenőriz duplikált e-mailt regisztrációkor — a mentés az adatbázis egyedi kényszere alapján dől el.
- A jelszót a rendszer soha nem tárolja nyílt szöveggént. A PasswordHasher<object>.HashPassword() metódus PBKDF2 alapú, szózott hash értéket készít a jelszóból, amelyet a Users tábla PasswordHash mezője tárol.
- Sikeres mentés után a rendszer lekérdezi az újonnan létrehozott felhasználót e-mail cím alapján, automatikusan RoleId = 1 (user) szerepkört rendel hozzá, majd létrehoz egy üres, aktív kosarat (Cart) a felhasználó azonosítójával. A végpont 204 NoContent választ ad vissza.

Bejelentkezési folyamat:

- A felhasználó megadja az e-mail címét és jelszavát (LoginRequest DTO). A backend a Users táblában keres e-mail cím alapján, és Include(u => u.Role) segítségével egyszerre betölti a szerepkör adatait is. Ha a felhasználó nem létezik, azonnal 401 Unauthorized választ küld.
- A PasswordHasher<object>.VerifyHashedPassword() metódus összehasonlítja a beküldött jelszót a tárolt hash-sel. Ha az eredmény PasswordVerificationResult.Failed, a rendszer 401 Unauthorized választ küld az "Invalid email or password." üzenettel — azonos hibaüzenet jelenik meg mind a nem létező e-mail, mind a helytelen jelszó esetén, hogy ne lehessen következtetni a hiba okára.

- Sikeres hitelesítés esetén a `GenerateJwtToken()` metódus állít ki JWT token. A token claims mezői: `userId`, e-mail cím, `firstName`, `lastName` és szerepkör neve. Az aláíráshoz HMAC-SHA256 algoritmust használ, a titkos kulcsot az `appsettings.json` `JwtOptions` szekciójából olvassa. A kész token a szerver `HttpOnly`, `Secure`, `SameSite=Lax` beállítású sütiben (cookie-ban) küldi vissza — nem `Authorization` fejlécben —, így kliensoldalon JavaScript nem férhet hozzá.
- A rendszer két további végpontot biztosít: a `GET /auth/cookieToken` a sütiben tárolt tokenet adja vissza (csak hitelesített felhasználóknak), a `POST /auth/logout` pedig törli a sütit. A szerepkör megváltoztatását a `PUT /auth/changeRole` végpont kezeli, amely e-mail cím és új `RoleId` alapján frissíti a felhasználó jogosultságát.

4.2 Felhasználói szerepkör módosítása

A rendszer lehetőséget biztosít a felhasználók jogosultsági szintjének módosítására. Ennek segítségével egy adott felhasználó szerepköre normál felhasználóról (User) adminisztrátorra (Admin) módosítható, illetve vissza is állítható.

A szerepkörök az alábbi módon vannak definiálva:

- 1 – User (alapértelmezett felhasználó)
- 2 – Admin (rendszergazda jogosultságok)

Elérési útvonal

A szerepkör módosítása egy backend API végponton keresztül történik:

- HTTP metódus: PUT
- Endpoint: `/auth/changeRole`
- Elérés: Swagger felületen keresztül
- Port: `https://localhost:5000/swagger`
- A végpont a Swagger UI segítségével könnyen tesztelhető a böngészőből.

Működés leírása

A végpont egy `RoleChangeDto` objektumot vár bemenetként, amely tartalmazza:

- a felhasználó e-mail címét (Email)
- az új szerepkör azonosítóját (RoleId)

Feldolgozás lépései:

1. A rendszer lekéri az aktuális időpontot (DateTime.Now).
2. Az adatbázisból megkeresi a felhasználót az e-mail cím alapján.
3. Betölti a felhasználóhoz tartozó jelenlegi szerepkört (Include(u => u.Role)).
4. Lekérdezi az összes létező szerepkör azonosítóját.
5. Ellenőrzi:
 - létezik-e a felhasználó
 - létezik-e a megadott szerepkör
6. Ha minden feltétel teljesül:
 - frissíti a felhasználó RoleId mezőjét
 - beállítja a módosítás idejét (ModifiedAt)
 - menti a változtatásokat az adatbázisba

Hibakezelés

A rendszer az alábbi eseteket kezeli:

- Felhasználó nem található
 - Válasz: 404 Not Found
 - Üzenet: "User not found."
- Nem létező szerepkör
 - Válasz: 404 Not Found
 - Üzenet: "Role not found."
- Sikeres válasz
 - Sikeres módosítás esetén:
 - HTTP státuszkód: 204 NoContent
 - A válasz nem tartalmaz adatot, csak a sikeres végrehajtást jelzi.

4.3 Kosár és rendeléskezelési folyamat

A CartsController kezeli a kosárral kapcsolatos műveleteket. A POST /carts/AddProduct végpont egyszerre látja el a hozzáadás, mennyiségmódosítás és eltávolítás funkcióját. Minden végpont JWT tokenből olvassa ki a bejelentkezett felhasználó azonosítóját (User.Claims, "userId" claim), így a kosár mindig az aktuális felhasználóhoz kötődik.

Termék hozzáadása a kosárhoz:

- A frontend POST kérést küld a /carts/AddProduct végpontra egy AddProductToCartDto objektummal (ProductId, Quantity). A backend betölti a

bejelentkezett felhasználó kosárát eager loading-gal: $\text{CartProducts} \rightarrow \text{Product} \rightarrow \text{ProductSpecs} \rightarrow \text{Spec}$ és Category .

- A rendszer lekérdezi a kosárban lévő termék-azonosítókat (`existingProductIds`). Ha a termék már szerepel: $\text{Quantity} \leq 0$ esetén törli a `CartItem` sort (eltávolítás), egyébként frissíti a mennyiséget. Ha a termék még nincs a kosárban: $\text{Quantity} \leq 0$ esetén 404-et ad vissza, egyébként új `CartItem` sort szúr be az adatbázisba.
- Mindkét esetben a végpont `201 CreatedAtAction` választ ad vissza, és a `MapToDto()` metódus a kosár tartalmát `CartItemReadDto`-vá alakítja: tartalmazza a kosárazonosítót, a felhasználó e-mail címét, és minden termékhez a mennyiséget, az árat, a speci-fikációkat és a kategória adatait.

Kosár lekérdezése:

- A `GET /carts/Current` végpont (`[Authorize]`) a JWT tokenből kiolvasott `userId` alapján kérdezi le a bejelentkezett felhasználó kosárát. A `GET /carts/{id}` végpont kosárazonosító alapján érheti el bárki a kosár adatait (például a rendelés megerősítése után), 404-et ad vissza, ha a kosár nem található.
- A `GET /carts` végpont az összes kosár adatait adja vissza (adminisztrációs célra). Mindkét lekérdező végpont ugyanazt a többszintű eager loading-ot alkalmazza: $\text{CartProducts} \rightarrow \text{Product} \rightarrow \text{ProductSpecs} \rightarrow \text{Spec}$, és az eredményt `CartItemReadDto`-ként adja vissza.
- Az `AddProduct` végpont rendelés-létrehozást nem végez — a kosár és a rendelés szétválasztott: a `CartsController` csak a kosár tartalmát kezeli, a rendelés véglegesítését az `OrdersController` végzi.
- A kosár adatai mindig valós időben tükröződnek: az `AddProduct` végpont minden művelet után visszaadja a teljes, frissített kosár tartalmát, így a frontend mindig konzisztens állapotot mutat a felhasználónak.

4.4 Rendelés létrehozása és életciklus-kezelés

A rendelés létrehozását a `POST /orders` végpont (`[Authorize]`) végzi az `OrdersController`-ben. A folyamat az `OrderCreateDto` adatait fogadja: szállítási cím (`AddressLine`, `City`, `ZipCode`) és fizetési mód (`PaymentMethod`), valamint a rendelés státusza (`Status`).

Rendelés létrehozásának lépései:

- A backend a JWT tokenből kinyeri a `loggedInUserId` értékét, majd betölti a felhasználó kosárát a `CartProducts` kapcsolattal együtt. Ha a kosár üres (`CartProducts.Count <= 0`), a rendszer 404 `NotFound` választ ad vissza.
- Érvényes kosár esetén létrejön egy új `Order` rekord a megadott szállítási adatokkal, fizetési móddal és `UserId`-vel. Ez kerül először az adatbázisba (`SaveChangesAsync`), hogy az új rendelés `Id`-je rendelkezésre álljon a következő lépéshez.
- Ezután egy `foreach` ciklus bejárja a kosár összes `CartProduct` elemét: minden elemhez létrehoz egy `OrderProduct` rekordot (mennyiséggel és `ProductId`-vel), majd törli az adott `CartProduct` sort. Mindez egyetlen `SaveChangesAsync` hívással kerül az adatbázisba — a kosár ezzel automatikusan kiürül.

Lejárat ellenőrzése:

- Rendelés lekérdezése és válasz:
- A mentés után a rendszer újra lekérdezi a létrehozott rendelést az adatbázisból eager loading-gal (`OrderProducts → Product → ProductSpecs → Spec`), hogy a válaszban a teljes termékadat szerepeljen. Az eredményt a `MapToDto()` alakítja `OrderReadDto`-vá, amely tartalmazza a szállítási adatokat, fizetési módot, státuszt és a rendelt termékek listáját mennyiségekkel.
- A végpont 201 `CreatedAtAction` választ ad vissza, amely a `GET /orders/{id}` végpontra mutat. A `GET /orders/Current ([Authorize])` a bejelentkezett felhasználó összes rendelését adja vissza. A `GET /orders/{id}` bárki számára elérhető, a `GET /orders` összes rendelést listáz (adminisztrációs célra).

Admin rendeléskezelés:

- Két `[Authorize(Roles = "Admin")]` végponttal kezelhető a rendelés életciklusa: a `PUT /orders/Status/{id}` frissíti a rendelés státuszát (pl. feldolgozás alatt, teljesítve), a `PUT /orders/Active/{id}` pedig az `Active` mezőt kapcsolja át (logikai törlés). Mindkét végpont ellenőrzi, hogy az URL-ben és a DTO-ban megadott azonosítók egyeznek, eltérés esetén 400 `BadRequest` választ ad. Sikeres módosítás esetén 204 `NoContent` választ küld vissza.

5. Tesztelések

5.1 Tesztelési stratégia

A rendszer fejlesztése során kiemelt figyelmet fordítottunk a különböző alkalmazásrétegek tesztelésére annak érdekében, hogy biztosítsuk a működés helyességét, a hibák minimalizálását és a stabil felhasználói élményt.

A tesztelést három fő szinten valósítottuk meg:

- Frontend tesztelés (React)
- Backend tesztelés (ASP.NET Core)
- Integrációs és felhasználói folyamatok ellenőrzése

5.2 Frontend tesztelés

A frontend alkalmazás tesztelése során a felhasználói felület működését és a felhasználói interakciókat vizsgáltuk.

Az alábbi főbb funkcionális egységek kerültek tesztelésre:

- Termékek listázása
- Termék részletek megjelenítése
- Kosár működése
- Rendelési folyamat
- Felhasználói regisztráció

5.3 Tesztelési típusok

A frontend oldalon az alábbi tesztípusokat alkalmaztuk:

- Komponens tesztek – az egyes UI elemek helyes megjelenítésének ellenőrzése
- Interakciós tesztek – felhasználói műveletek (pl. gombkattintás, űrlapkitöltés) vizsgálata
- Egységtesztek – segédfüggvények (utils) működésének ellenőrzése

Tesztelési eszközök

- React Testing Library
- Vitest / Jest (tesztfuttató környezet)
- Mockolt API válaszok (backend független teszteléshez)

5.4 Backend tesztelés

A backend (ASP.NET Core 8) tesztelése során az üzleti logika és az API végpontok helyességét ellenőriztük.

Tesztelt területek:

- Felhasználói hitelesítés (regisztráció, bejelentkezés)
- JWT token kezelés
- Kosár műveletek
- Rendelés létrehozás és lekérdezés
- CRUD műveletek (termékek, kategóriák)
- Teszt típusok
- Egységtesztek – szolgáltatások és üzleti logika vizsgálata
- Integrációs tesztek – API végpontok és adatbázis kapcsolat ellenőrzése

Tesztelési eszközök

- xUnit
- FluentAssertions
- ASP.NET Core TestServer / WebApplicationFactory

5.5 API tesztelés

Az API működésének ellenőrzése során külön figyelmet fordítottunk:

- HTTP státusz kódok helyességére (200, 201, 400, 401, 404, 500)
- Hibakezelésre
- Jogosultságkezelésre (Authorize attribútumok)
- Adatvalidációra

5.6 Tesztelési eredmények

A tesztelés során a rendszer fő funkciói megfelelően működtek. A kritikus felhasználói folyamatok (pl. kosár kezelés, rendelés leadás, bejelentkezés) stabilan és hibamentesen futnak.

A hibák feltárása és javítása iteratív módon történt a fejlesztési folyamat során.

IV. Felhasználói dokumentáció

6. Program általános specifikációja és célja

6.1 Program célja

Az alkalmazás egy modern e-kereskedelmi platform, amely lehetővé teszi a felhasználóknak, hogy online vásároljanak különféle elektronikai termékeket. A rendszer termékek böngészésére, szűrésére, kosárba helyezésére, valamint rendelések kezelésére biztosít funkcionálisitást.

7. Általános specifikáció

Alkalmazástípus: Webalapú e-kereskedelmi rendszer

Architektúra: Modern 3-rétegű alkalmazás (Frontend - React, Backend - ASP.NET Core, Database - SQLite)

7.1 Támogatott funkciók:

Felhasználói regisztráció és bejelentkezés

Termékek böngészése kategóriák szerint (telefon, notebook, kiegészítők)

Termék szűrés és keresés

Kosár kezelés

Felhasználói profil és rendelési előzmények

Admin funkciók (termékek, specifikációk, rendelések kezelése)

8. Rendszer követelmények

8.1 Szoftver követelmények

- Operációs rendszer
- Windows 10 vagy újabb (Windows 11 ajánlott)

8.2 Backend követelmények

- .NET SDK 8.0 vagy újabb
- SQL Server, PostgreSQL, vagy SQLite adatbázis (projekt SQLite-ot használ)

8.3 Frontend követelmények

- Node.js 16.0 vagy újabb verzió
- Bármelyik modern webböngészőt használható:
 - Google Chrome (ajánlott)
 - Microsoft Edge
 - Mozilla Firefox
 - Safari
 - Egyéb ES2020+ támogatással rendelkező böngészők

8.4 Hardver követelmények

- Processzor: Intel Core i5 / AMD Ryzen 5 vagy jobb
- RAM: Minimum 4 GB (8 GB ajánlott)
- Szabad lemezterület: Minimum 1 GB
- Internetkapcsolat: Helyi hálózat / localhost teszteléshez

8.5 Hálózati portok

- Frontend: localhost:5000
- Backend API: localhost:5000
- Swagger UI (API dokumentáció): localhost:5000/swagger

9. Program telepítése

9.1 Előfeltételek ellenőrzése

Ellenőrizze, hogy az alábbi szoftverek telepítve vannak powershellben:

- **.NET verzió ellenőrzése**
 - dotnet --version
- **Node.js verzió ellenőrzése**
 - node --version

9.2 Projekt letöltése

A projektmappát töltsse le vagy klónozza Git-ből:

Powershellben:

- git clone https://github.com/AronFLS/2025-26_Vizsgaremek.git
- cd 2025-26_Vizsgaremek

9.3 Az alkalmazás indítása

- Telepítse a Node.js-t, ha még nincs telepítve ([Node.js — Download Node.js®](#))
- Telepítsen .Net 8 vagy magasabb verziót, ha még nincs telepítve ([Download .NET \(Linux, macOS, and Windows\) | .NET](#))
- Indítsa el az „AppleTopia.exe” fájlt a projektmappából

9.4 Indítási adatok

Az alkalmazás az adatbázis inicializálása során beépített Roleokat, kategóriákat Productokat és Specifikációkat hoz létre. A Roleokból létrehoz egy „user” (RoleId = 1) és egy „admin” (RoleId = 2). Kategóriákból létrehoz egy „phone”, „macbook” és egy „accessories” kategóriát. Productok és Specifikációk részletezéséhez nézze meg a „SeedData.cs” fájlt a „Backend/Backend” mappában

10. Program használata

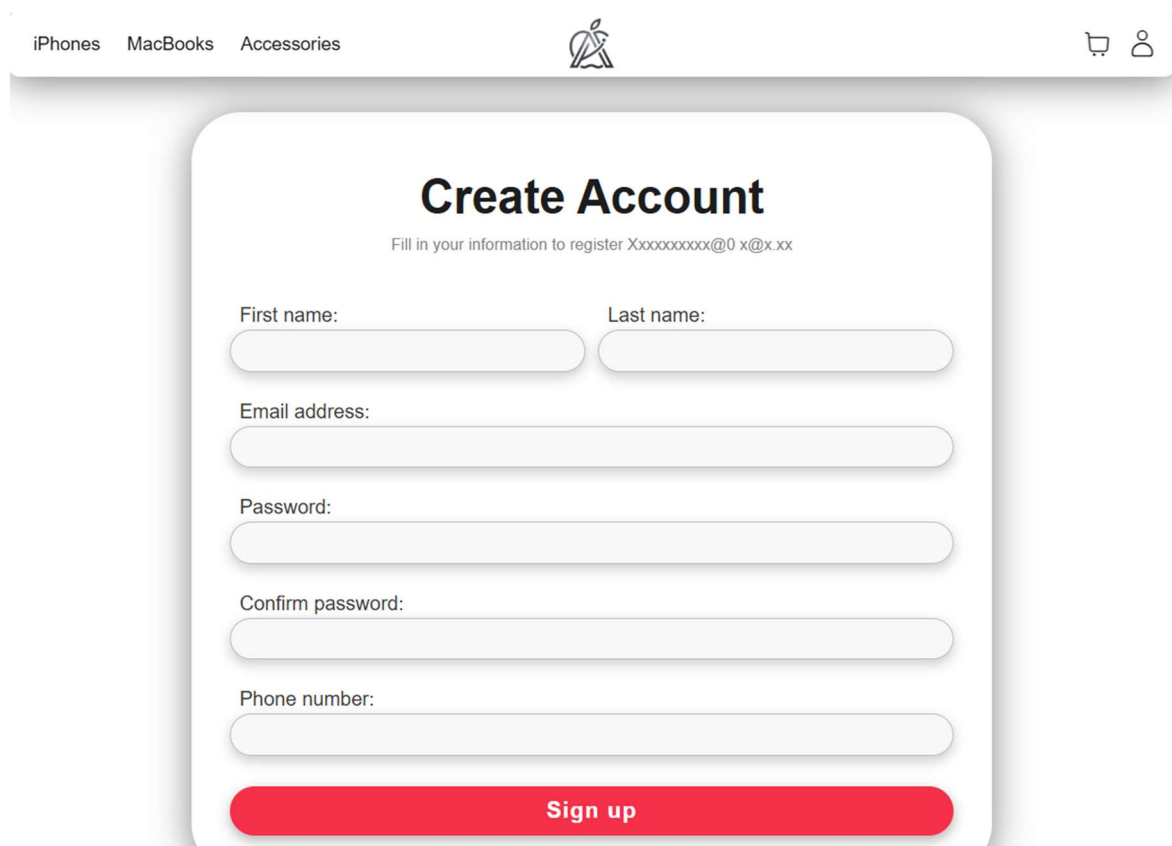
10.1 Az alkalmazás indítása

Ha elindította a fájlt, akkor a webshop fő oldalán fogja magát találni a „localhost:5000”-es porton

10.2 Felhasználói fiókok

10.2.1 Regisztráció

- Kattintson a felhasználó ikonra
- Nyomjon a „Sign up” piros szövegre
- Adja meg az adatokat (biztonságos jelszót adjon meg)

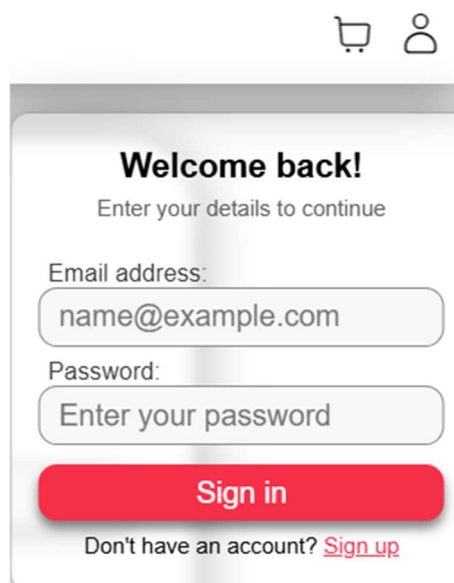


The image shows a 'Create Account' form on a website. At the top, there is a navigation bar with links for 'iPhones', 'MacBooks', and 'Accessories', a logo, and icons for a shopping cart and user profile. The form itself is titled 'Create Account' and includes a subtext: 'Fill in your information to register Xxxxxxxxxx@0 x@x.xx'. It contains several input fields: 'First name:', 'Last name:', 'Email address:', 'Password:', 'Confirm password:', and 'Phone number:'. Each field is represented by a light gray rounded rectangle. At the bottom of the form is a prominent red button labeled 'Sign up'.

2. ábra Regisztrációs oldal

10.2.2 Bejelentkezés

- Kattintós a felhasználó ikonra
- Írja be az adatokat
- Kattintson a „Sign in” piros gombra



The image shows a 'Sign in' form on a website. At the top, there is a navigation bar with icons for a shopping cart and user profile. The form is titled 'Welcome back!' and includes a subtext: 'Enter your details to continue'. It contains two input fields: 'Email address:' with the placeholder text 'name@example.com' and 'Password:' with the placeholder text 'Enter your password'. Both fields are represented by light gray rounded rectangles. At the bottom of the form is a prominent red button labeled 'Sign in'. Below the button, there is a link: 'Don't have an account? [Sign up](#)'.

3. ábra Bejelentkezési fül

10.3 Termékek böngészése

A menüben válasszon egy kategóriát:

- Telefonok
- MacBookok
- Kiegészítők

iPhones MacBookok Accessories



4. ábra Menü választási opciói

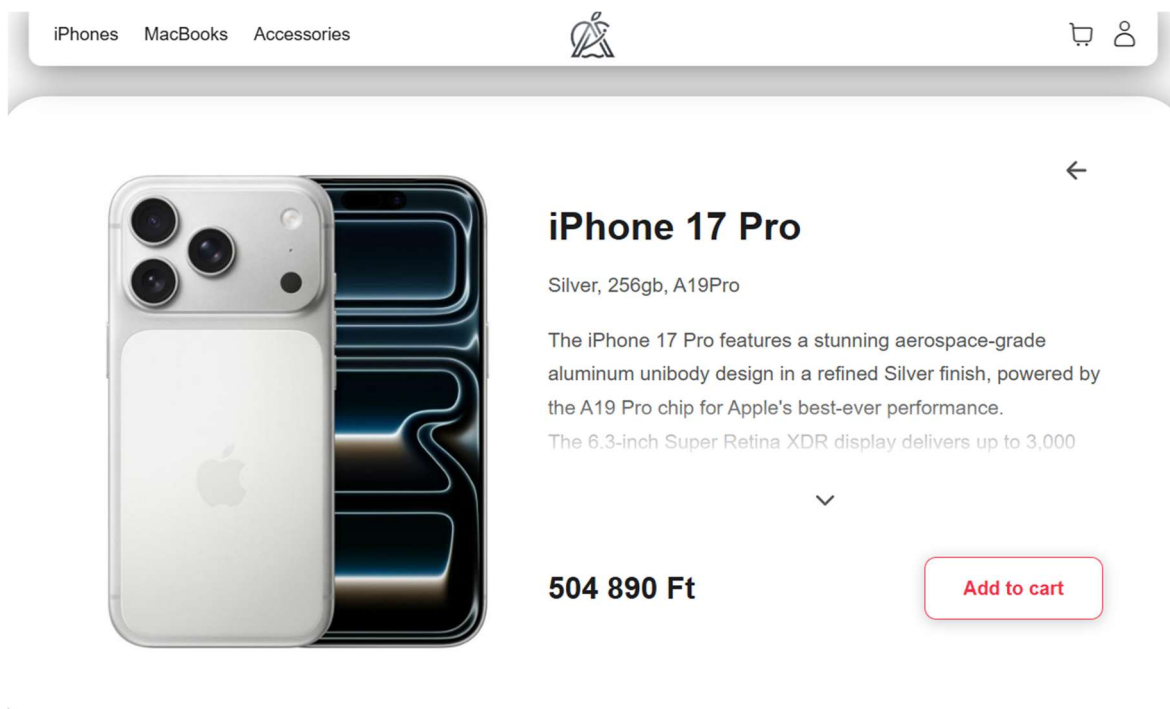
Megjelennek az adott kategóriában lévő termékek

10.4 Termék részletei

Kattintson egy termékre a részleteket tekintéséhez

Megtekintheti:

- Terméknevet és leírást
- Árat (ha le van árazva akkor a leárazott ár látható)
- Specifikációkat (paraméterek)



5. ábra Egy termék oldala

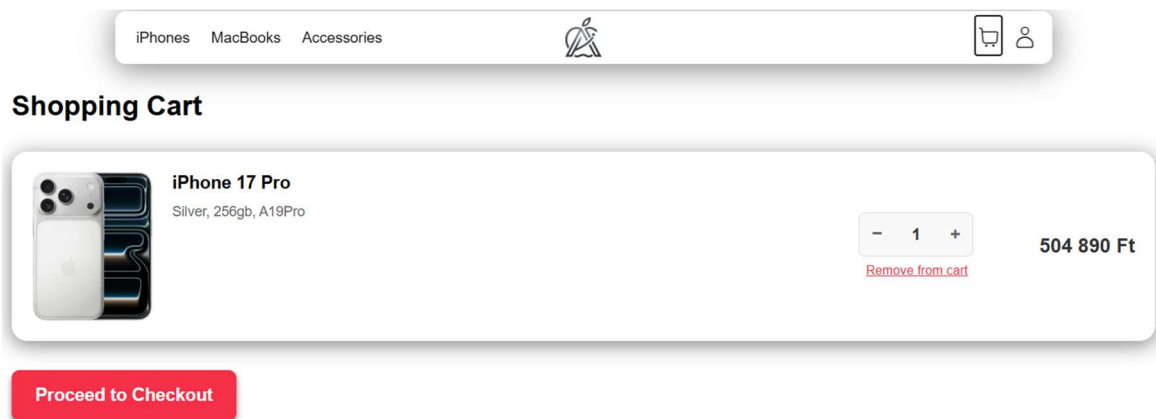
10.5 Kosár kezelés

10.5.1 Termék hozzáadása a kosárhoz

- Válasszon egy terméket
- Nyomja meg az „Add to cart” gombot
- A termék hozzáadódik a kosárhoz

10.5.2 Kosár megtekintése

- Kattintson a Kosár ikonra (felhasználó ikon mellett)
- Megtekintheti:
 - A kosárban levő termékeket
 - Az egyes termékek mennyiségét és árát
 - A teljes végösszeg



6. ábra Kosár

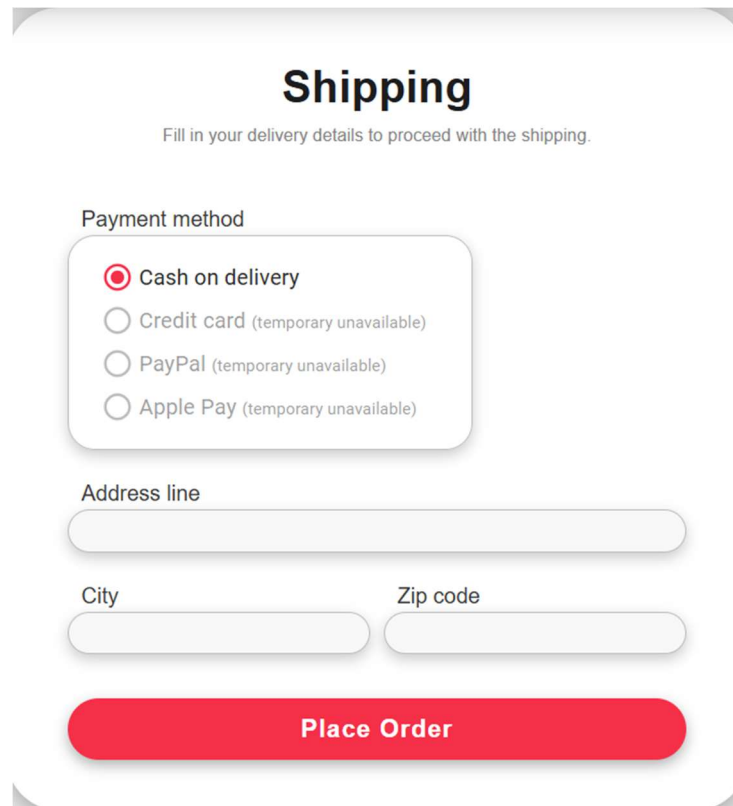
10.5.3 Termék eltávolítása a kosárból

- A kosárban keresse meg az eltávolítandó terméket
- Kattintson a „Remove from cart” piros szövegre
- A termék törlődik a kosárból

10.5.4 Rendelés leadása

- Kattintson a „Proceed to Checkout” gombra a kosárban
- Adja meg a szállítási adatokat:
 - Lakcím
 - Város

- Zip kód
- Válassza ki a fizetési módot (jelenleg csak Cash on delivery)
- Nyomjon a „Place Order” gombra



The image shows a 'Shipping' form with a title 'Shipping' and a subtitle 'Fill in your delivery details to proceed with the shipping.' Below this is a 'Payment method' section with four radio buttons: 'Cash on delivery' (selected), 'Credit card (temporary unavailable)', 'PayPal (temporary unavailable)', and 'Apple Pay (temporary unavailable)'. Below the payment methods is an 'Address line' input field, followed by 'City' and 'Zip code' input fields. At the bottom is a large red 'Place Order' button.

Shipping

Fill in your delivery details to proceed with the shipping.

Payment method

☒ Cash on delivery

☐ Credit card (temporary unavailable)

☐ PayPal (temporary unavailable)

☐ Apple Pay (temporary unavailable)

Address line

City Zip code

Place Order

7. ábra Szállítási adatok oldala

- Checkoutnál utoljára átnézheti az adatokat
- Nyomjon az „Order” gombra
- Kap egy emailt, hogy a rendszer megkapta a rendelésed

iPhoneMacBooksAccessories





Checkout

Your Information

Name: user user

Email: user

Phone number: user

Shipping

Payment: Cash on delivery

Address: 1234, city, addressa line

Ordered Products



iPhone 17 Pro

Silver, 256gb, A19Pro

Quantity: 1

504 890 Ft

Total

504 890 Ft

Order

8. ábra Checkout oldal

10.5.5 Rendelések követése

- Kattintson a felhasználó ikonra
- Kattintson az „Account details” piros szövegre
- Megtekintheti az aktív rendeléseit és a rendelési előzményeit és az ezekhez tartozó információkat (fizetési mód, szállítási cím, termékek, összeg)

Active Orders

Track your ongoing orders and check their status.

Order #3

1 product(s)

Payment: Cash on delivery

View details

pending

Past Orders

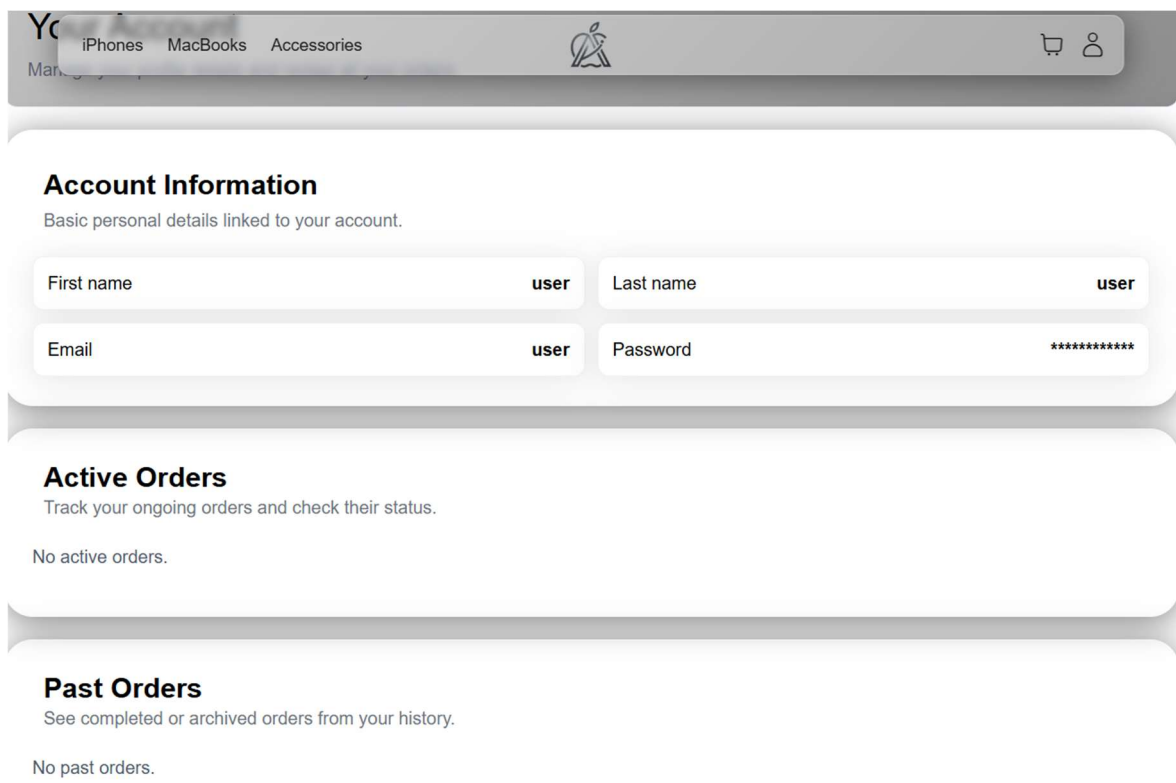
See completed or archived orders from your history.

No past orders.

9. ábra Rendelések megtekintése

10.6 Felhasználói fiók kezelése

- Kattintson a felhasználó ikonra
- Válassza az "Account details"
- Itt megtekintheti
 - Személyes adatokat
 - Email címet
 - Jelszót
 - Rendeléseket



Yc iPhones MacBooks Accessories

Account Information

Basic personal details linked to your account.

First name	user	Last name	user
Email	user	Password	*****

Active Orders

Track your ongoing orders and check their status.

No active orders.

Past Orders

See completed or archived orders from your history.

No past orders.

10. ábra Felhasználói oldal

10.7 Admin funkciók (adminisztrátorok számára)

10.7.1 Admin panel elérése

- Jelentkezzen be admin fiókkal
- Kattintson az "Admin" menüpontra



iPhones MacBooks Accessories

Admin

11. ábra Menü Admin felhasználóval

10.7.2 Termékek kezelése

10.7.2.1 Új termékek hozzáadása

- Válassza ki a „PostProduct/Spec” menü pontot
- Adja meg a Product információit (Ha tölt fel egy képet és egy linket is rak be, akkor a feltöltött kép lesz prioritizálva és azt fogja a program a továbbiakban használni)

Post Product/Spec ActiveOrders PastOrders UpdateProducts

Add Product

NAME

Product name

PRODUCT IMAGE URL

https://example.com/image.jpg

OR UPLOAD IMAGE FILE

Choose File No file chosen

PRICE

0

DISCOUNT (%)

0

STOCK QUANTITY

0

DESCRIPTION

Product description

CATEGORY

Select category

12. ábra Termék hozzáadása oldal

10.7.2.2 Meglévő termékek kezelése

- Kattintson az „UpdateProducts” menüpontra
- Itt lehet megváltoztatni egy adott termék árát, leárazását, darabszámát és státuszát (Csak aktív státuszú termékek jelennek meg a felhasználónak)

13. ábra Termék frissítése oldal

10.7.3 Specifikációk kezelése

- Válassza a "PostProduct/Spec" menüpontot
 1. Görgessen le a lap aljára
 2. Írja be a kívánt specifikációt
 3. Nyomjon az „Add Spec” gombra

Add Spec

SPEC NAME

e.g. 1TB, M5, SkyBlue

Add Spec

14. ábra Specifikáció hozzáadása

10.7.4 Rendelések kezelése

- Válassza ki az "ActiveOrders" menüpontot
- Megtekintheti az összes rendelést és változtathatja azok státuszát
- Ha egy rendelés státusza „Delivered” vagy „Cancelled”, akkor átkerülnek a PastOrders menüpont alá

Post Product/Spec
ActiveOrders
PastOrders
UpdateProducts

Active Orders

Search by order id, user id, status...

2 active order(s)

Order #2

Pending

User ID: 1

Payment: Cash on delivery

Address: 1351, asd, 235asd

MacBook Air M5 x 1472690Ft

MagSafe Charger 2m x 119890Ft

Total: 492580Ft

Change status

Pending

Pending

Processing

Shipping

Delivered

Cancelled

Order #3

Pending

User ID: 2

Payment: Cash on delivery

Address: 1234, city, address line

iPhone 17 Pro x 1504890Ft

Total: 504890Ft

Change status

Pending

Update

15. ábra Rendelés kezelése oldal

10.8 Ügyfélszolgálat és Hiba kezelés

Gyakori hibák és megoldásaik

- "Nem sikerült csatlakozni az API-hoz" - Ellenőrizze, hogy az alkalmazás fut-e a `http://localhost:5000` címen
- "A bejelentkezés sikertelen" - Ellenőrizze az email és jelszó helyességét
- "Üres oldal jelenik meg" - Frissítse az oldalt (F5), vagy törölje a böngésző gyorsítótárát
- "Termékek nem töltődnek be" - Ellenőrizze az internet kapcsolatot és az adatbázist

V. Összegzés

A vizsgaremek elkészítése során sikeresen megvalósítottunk egy korszerű, több rétegből felépülő webáruház rendszert, amely megfelel a modern e-kereskedelmi alkalmazásokkal szemben támasztott alapvető követelményeknek. A fejlesztés során olyan technológiákat alkalmaztunk, amelyek napjainkban széles körben elterjedtek a szoftverfejlesztés területén, ezáltal gyakorlati tapasztalatot szereztünk mind frontend, mind backend oldalon.

A projekt során megismerkedtünk a kliens–szerver architektúra működésével, az adatbázis-tervezés folyamatával, a REST API alapú kommunikációval, valamint a hitelesítés és jogosultságkezelés megvalósításával. Emellett fejlesztettük együttműködési készségeinket is, hiszen a csoportmunka során fontos szerepet kapott a feladatok megfelelő elosztása, a verziókezelés használata és a folyamatos egyeztetés.

Az elkészült rendszer lehetőséget biztosít a felhasználók számára a regisztrációra, bejelentkezésre, termékek böngészésére, kosárkezelésre és rendelések leadására, míg az adminisztrátori felület támogatja a termékek, specifikációk és rendelések kezelését. A tesztelések eredményei alapján a rendszer stabilan működik, a fő funkciók megbízhatóan használhatók.

További jövőbeli fejlesztési lehetőségek

Az alábbi fejlesztések megvalósításával a rendszer még közelebb kerülhetne egy valós környezetben működő, professzionális webáruház szintjéhez. Ezek a bővítések növelnék a felhasználói élményt, javítanák az értékesítési lehetőségeket, valamint korszerűbbé tennék a szolgáltatást.

Kedvezménykuponok és promóciók

A kuponrendszer bevezetésével lehetőség nyílna különböző kedvezmények biztosítására a vásárlók számára. Ide tartozhatnak százalékos vagy fix összegű akciók, időszakos kampányok, valamint első vásárlóknak szóló kedvezmények.

Bankkártyás online fizetés

A jelenlegi fizetési módok mellett célszerű lenne bankkártyás fizetési lehetőséget is biztosítani. Ez gyorsabbá, kényelmesebbé és modernebbé tenné a rendelési folyamatot a felhasználók számára.

Lapozható terméklista és többoldalas megjelenítés

Nagyobb termékkínálat esetén indokolt a termékek több oldalon történő megjelenítése. A lapozható lista javítaná az átláthatóságot, gyorsabb betöltést biztosítana, valamint egyszerűbb navigációt nyújtana a felhasználóknak.

Termékváltozatok kezelése

Hasznos fejlesztés lenne a termékvariációk támogatása, például különböző színek, tárhelyméretek vagy modellek kiválasztási lehetőségével.

Hűségprogram és vásárlói jutalmazás

A visszatérő vásárlók számára kialakítható lenne egy pontgyűjtő vagy kedvezményalapú hűségprogram. Ez ösztönözné az ismételt vásárlásokat, és erősítené a vásárlói elégedettséget, valamint a márkához való kötődést.

Összefoglalás

A vizsgaremek elkészítése során sikeresen létrehoztunk egy modern felépítésű webáruház rendszert, amely elektronikai termékek online értékesítésére alkalmas. A frontend React, a backend ASP.NET Core technológiával készült, az adatkezelést pedig SQLite adatbázis biztosítja.

A rendszer tartalmazza a legfontosabb webshop funkciókat, mint a regisztráció, bejelentkezés, termékböngészés, kosárkezelés, rendelés leadása, valamint az adminisztrátori kezelőfelület.

A projekt során értékes tapasztalatot szereztünk a webfejlesztés, az adatbázis-kezelés, az API kommunikáció és a csapatmunka területén. Emellett megismertük a tesztelés és a verziókezelés fontosságát is.

Összességében a vizsgaremek sikeresen bemutatja megszerzett tudásunkat, és megfelelő alapot nyújt a későbbi fejlesztésekhez.

VI. Irodalomjegyzék, hivatkozásjegyzék

Inspiráció:

- iSTYLE: <https://istyle.hu/>

Backend fejlesztés:

- Microsoft Learn: [Microsoft Learn: Építsen úgy, hogy a válaszok kéznél legyenek](#)
- Stack Overflow: <https://stackoverflow.com/>

Frontend fejlesztés:

- React Icons: <https://react-icons.github.io/react-icons/>
- Material UI: <https://mui.com/material-ui/>
- Embla Carousel: <https://www.embla-carousel.com/>
- CSS Loaders: <https://css-loaders.com/dots/>

VII. Ábrajegyzék

1. ábra Adatbázis felépítése.....	11
2. ábra Regisztrációs oldal.....	23
3. ábra Bejelentkezési fül.....	23
4. ábra Menü választási opciói	24
5. ábra Egy termék oldala.....	24
6. ábra Kosár.....	25
7. ábra Szállítási adatok oldala	26
8. ábra Checkout oldal.....	27
9. ábra Rendelések megtekintése.....	27
10. ábra Felhasználói oldal	28
11. ábra Menü Admin felhasználóval.....	28
12. ábra Termék hozzáadása oldal.....	29
13. ábra Termék frissítése oldal.....	30
14. ábra Specifikáció hozzáadása	30
15. ábra Rendelés kezelése oldal	31